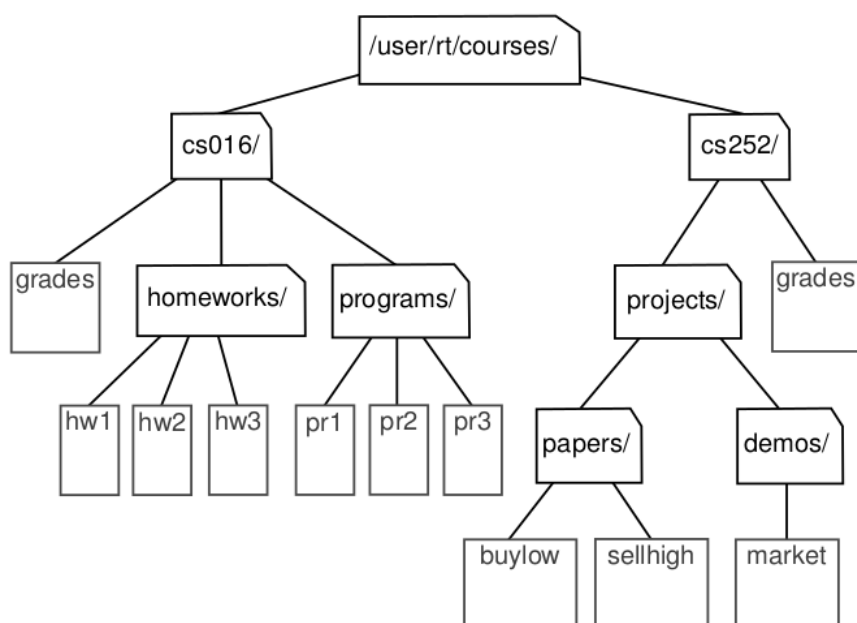


Trees

Applications – File directory



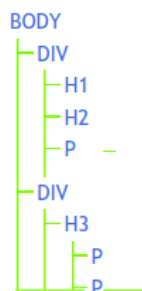
HTML & DOM tree

The browser parses the HTML and creates a tree of objects like this:

```
html
  head
    title
  body
    h1
    p
    script
```

```
<!doctype html>
<html>
  <head>
    <title>Hello, world!</title>
  </head>
  <body>
    <h1>Hello, world!</h1>
    <p>This is a hypertext document on the World Wide Web.</p>
    <script src="/script.js" async></script>
  </body>
</html>
```

```
1 <!DOCTYPE html>
2 <html>
3   <body>
4
5     <div>
6       <h1>DSA 2024</h1>
7       <h2>Welcome to the class.</h2>
8       <p> Well, actually not welcome as we have completed half way through </p>
9     </div>
10
11    <div>
12      <h3> So far we have covered many data structures and ADTS
13      <p> today we discuss trees </p>
14      <p> yet to discuss are : priority queues, Hash maps etc </p>
15    </h3>
16  </div>
17 </body>
18 </html>
```



DSA 2024

Welcome to the class.

Well, actually not welcome as we have completed half way through

So far we have covered many data structures and ADTS

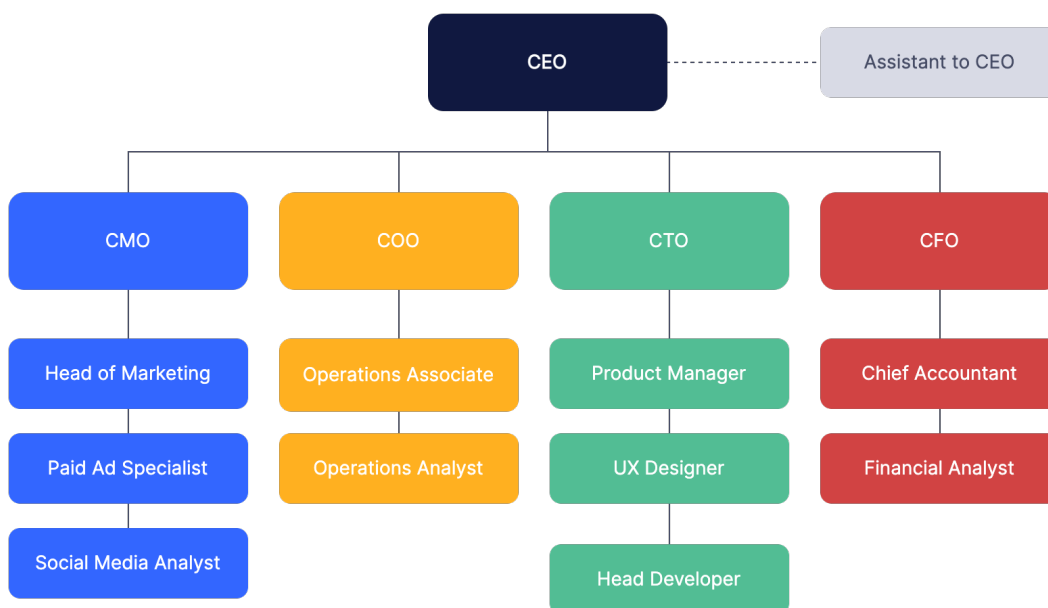
today we discuss trees

yet to discuss are : priority queues, Hash maps etc

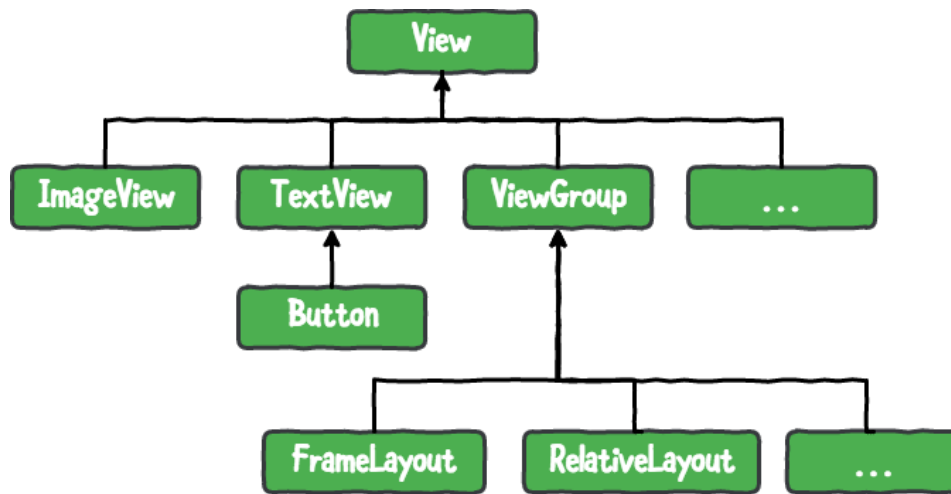
DOM tree

```
DOCTYPE: html
HTML
  HEAD
  BODY
    #text:
    DIV
      #text:
      H1
        #text: DSA 2024
      #text:
      H2
        #text: Welcome to the class.
      #text:
      P
        #text: Well, actually not welcome as we have completed half way through
      #text:
      #text:
      DIV
        #text:
        H3
          #text: So far we have covered many data structures and ADTS
        P
          #text: today we discuss trees
        #text:
        P
          #text: yet to discuss are : priority queues, Hash maps etc
        #text:
        #text:
```

Organisation structure

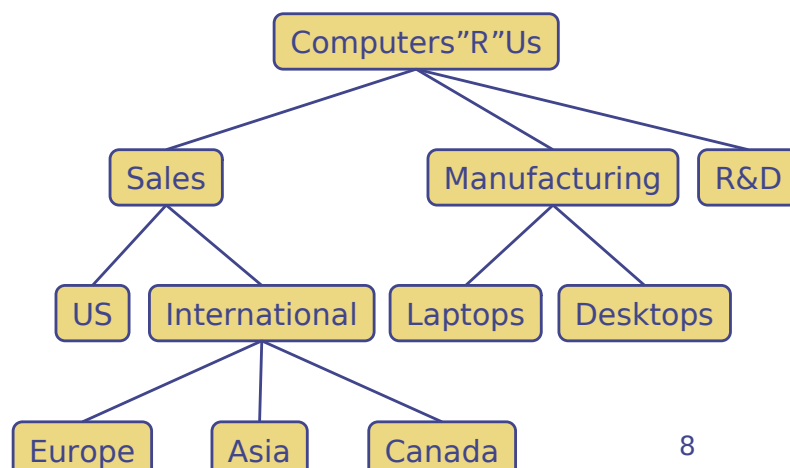


UI Layouts



What is a Tree?

- Tree is used to represent hierarchical relationships between elements. It consists of a collection of nodes connected by edges signifying parent-child relationship.
- A node can have zero or more children. Each node has at most one parent node except the root node which has none.



Applications summary

- File Systems
- Family Trees
- Social Networks
- Syntax Trees in Compilers
- UI layouts
- DOM trees to represent webpages
- Decision Trees in Machine Learning

9

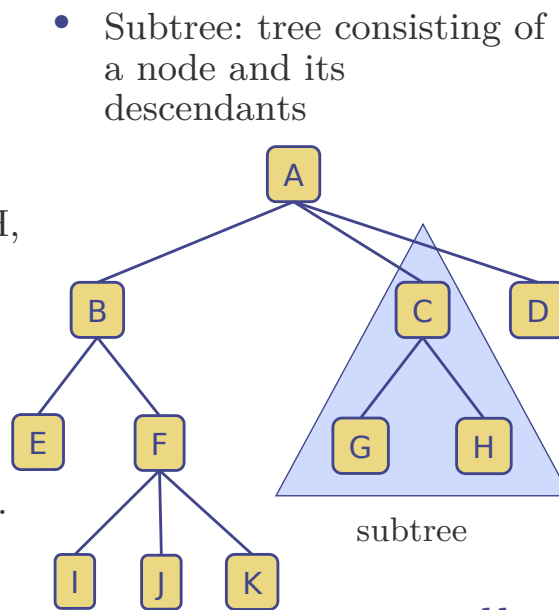
Example of a Syntax tree

$5 \times (6 + 9) - 3$:



Tree Terminology

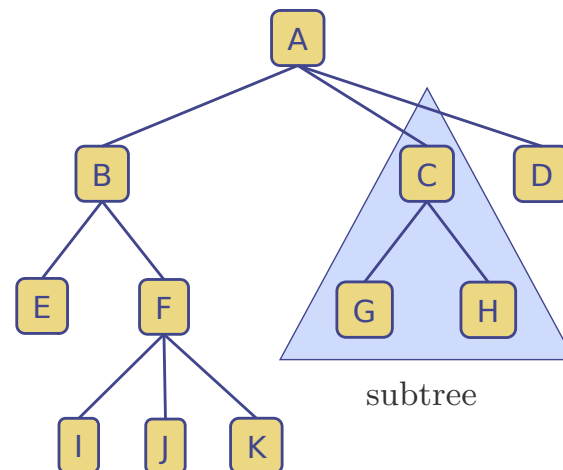
- Root: node without parent (A)
- Internal node: node with at least one child (A, B, C, F)
- External node (a.k.a. leaf): node without children (E, I, J, K, G, H, D)
- Ancestors of a node: parent, grandparent, grand-grandparent, etc.
- Descendant of a node: child, grandchild, grand-grandchild, etc.



11

Tree Terminology

- $\text{DEPTH}(p)$: The number of edges on the path from the root to vertex p (the depth of the root is 0).
- $\text{HEIGHT}(p)$: The number of edges on a longest path from vertex p to a leaf (leaves have height 0)
- $\text{LEVEL}(p)$: $\text{DEPTH}(p) + 1$
- Subtree: tree consisting of a node and its descendants



12

Tree ADT

A tree ADT is generally defined using the concept of a position as an abstraction for a node of a tree.

A position object for a tree supports the method:

`p.element()`: Return the element stored at position `p`

Generic methods:

Integer `len()`

Boolean `is_empty()`

Iterator `positions()`

Iterator `iter()`

Accessor methods:

position `root()`

position `parent(p)`

Iterator `children(p)`

Integer `num_children(p)`

13

Tree ADT

Query methods:

Boolean `is_leaf(p)`

Boolean `is_root(p)`

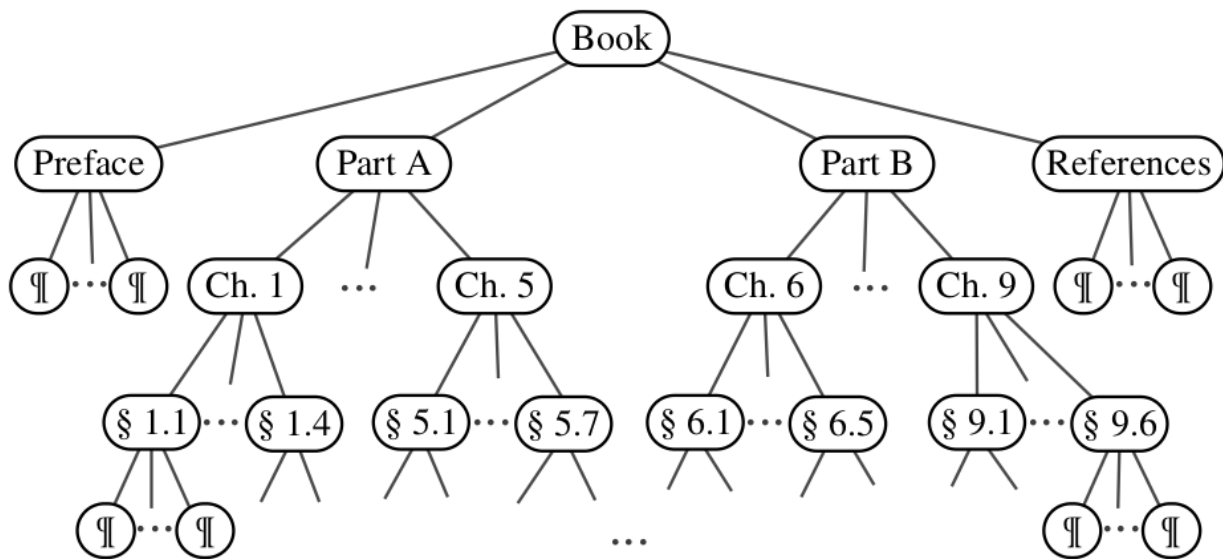
Update method:

element `replace(p, o)`

Additional update methods
may be defined by data
structures implementing the
Tree ADT

14

Ordered Tree example



Tree Traversal Algorithms

Preorder Traversal

A traversal visits the nodes of a tree in a systematic manner

In a preorder traversal, a node is visited before its descendants

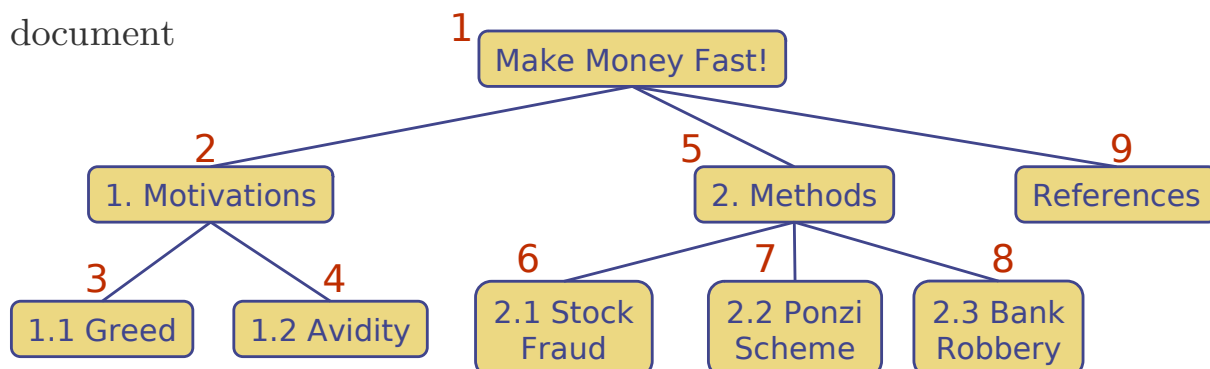
Application: print a structured document

Algorithm *preOrder*(*v*)

visit(*v*)

for each child *w* of *v*

preorder (*w*)

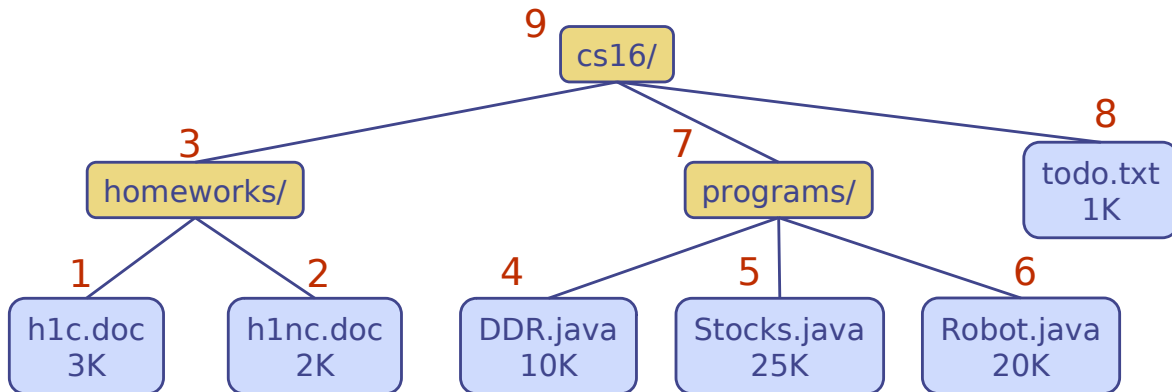


Postorder Traversal

In a postorder traversal, a node is visited after its descendants

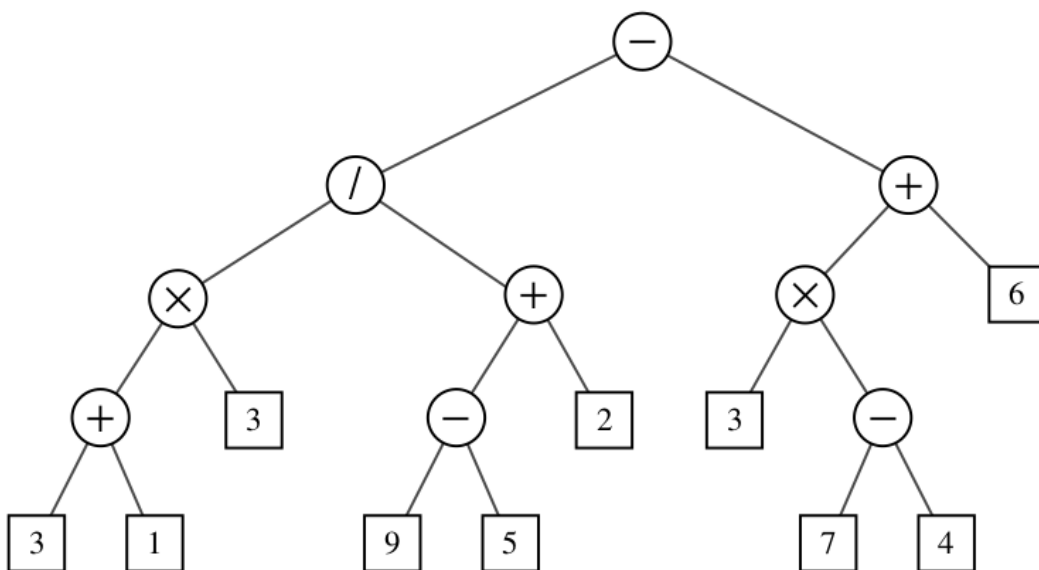
Application: compute space used by files in a directory and its subdirectories

Algorithm *postOrder*(*v*)
 for each child *w* of *v*
 postOrder (*w*)
 visit(*v*)



17

Binary Tree



Binary Trees

A binary tree is a tree with the following properties:

Each internal node has at most two children (exactly two for **proper** binary trees)

The children of a node are an ordered pair

We call the children of an internal node **left child** and **right child**

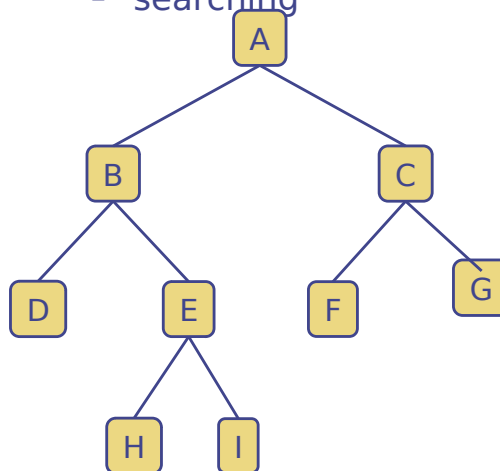
Alternative recursive definition: a binary tree is either

a tree consisting of a single node, or

a tree whose root has an ordered pair of children, each of which is a binary tree

Applications:

- arithmetic expressions
- decision processes
- searching



19

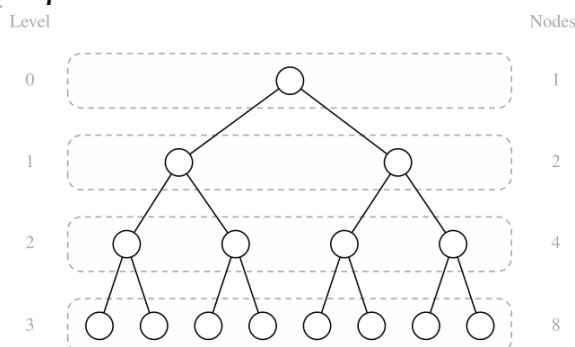
Binary Trees Properties

Proposition 8.8: Let T be a nonempty binary tree, and let n , n_E , n_I and h denote the number of nodes, number of external nodes, number of internal nodes, and height of T , respectively. Then T has the following properties:

1. $h + 1 \leq n \leq 2^{h+1} - 1$
2. $1 \leq n_E \leq 2^h$
3. $h \leq n_I \leq 2^h - 1$
4. $\log(n + 1) - 1 \leq h \leq n - 1$

Also, if T is proper, then T has the following properties:

1. $2h + 1 \leq n \leq 2^{h+1} - 1$
2. $h + 1 \leq n_E \leq 2^h$
3. $h \leq n_I \leq 2^h - 1$
4. $\log(n + 1) - 1 \leq h \leq (n - 1)/2$



Proper/Full Binary Tree vs Complete Binary Tree

Properties	Proper Binary Tree	Complete Binary Tree
Node Children	0 or 2	0, 1, or 2
Node Placement	No specific order	Nodes are as left as possible
Last Level	May not be completely filled	May not be completely filled

Note: there is also fully balanced binary tree where all its internal nodes have exactly two children and leaves are at the same depth or level.

Properties of Proper Binary Trees

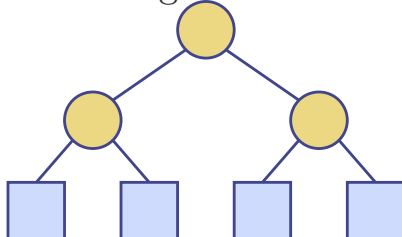
Notation

n number of nodes

e number of external nodes

i number of internal nodes

h height



◆ Properties:

- $e = i + 1$

- $n = 2e - 1$

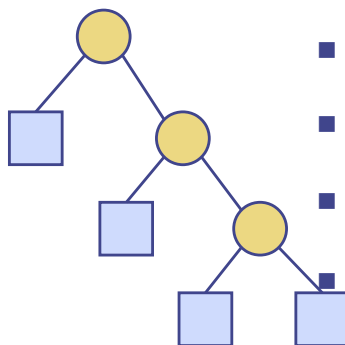
- $h \leq i$

- $h \leq (n - 1)/2$

- $e \leq 2^h$

- $h \geq \log_2 e$

- $h \geq \log_2 (n + 1) - 1$



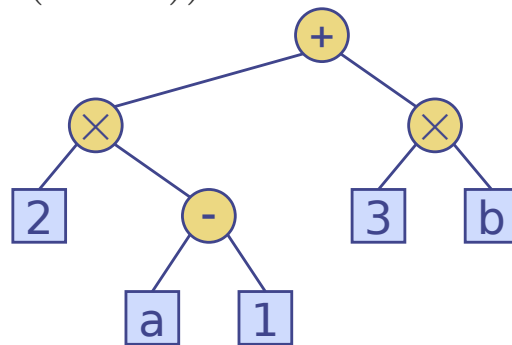
Arithmetic Expression Tree

Binary tree associated with an arithmetic expression

internal nodes: operators

external nodes: operands

Example: arithmetic expression tree for the expression
 $(2 \times (a - 1) + (3 \times b))$



23

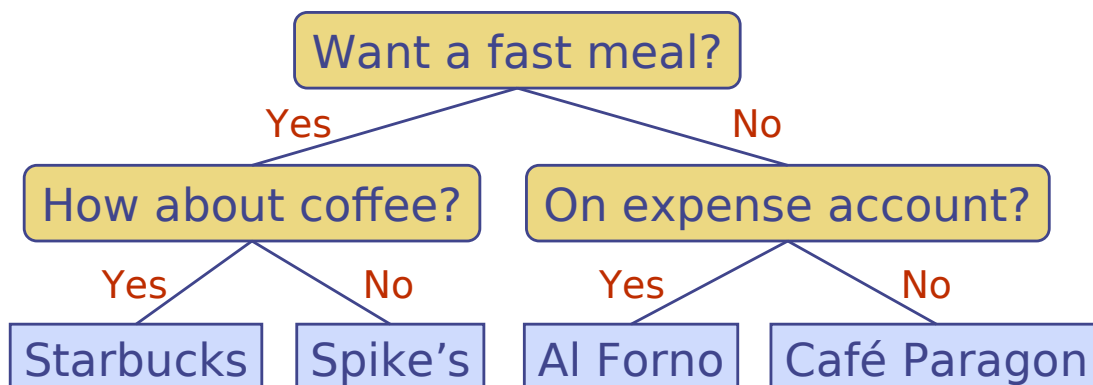
Decision Tree

Binary tree associated with a decision process

internal nodes: questions with yes/no answer

external nodes: decisions

Example: dining decision



24

BinaryTree ADT

The BinaryTree ADT extends the Tree ADT, i.e., it inherits all the methods of the Tree ADT

Update methods may be defined by data structures implementing the BinaryTree ADT

Additional methods:

position left(p)

position right(p)

position sibling(p)

25

Inorder Traversal

In an inorder traversal a node is visited after its left subtree and before its right subtree

Application: draw a binary tree

$x(v)$ = inorder rank of v

$y(v)$ = depth of v

Algorithm *inOrder*(v)

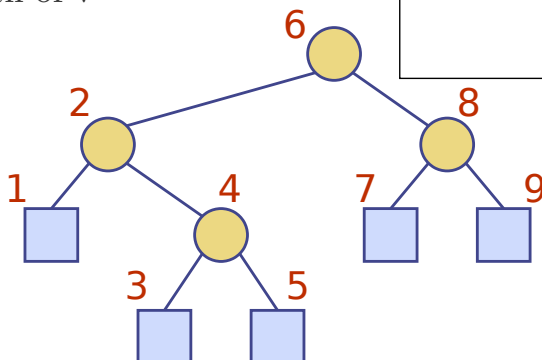
if v has a left child

inOrder (*left* (v))

visit(v)

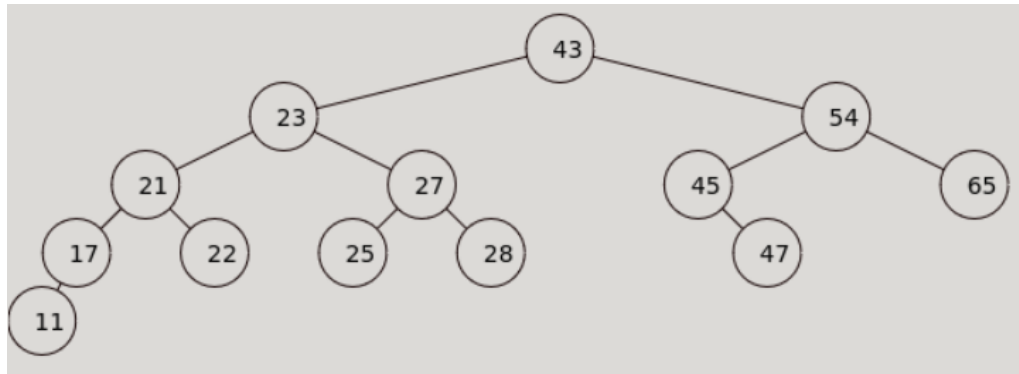
if v has a right child

inOrder (*right* (v))



26

Example – display elements using traversals



Preorder: 43 23 21 17 11 22 27 25 28 54 45 47 65

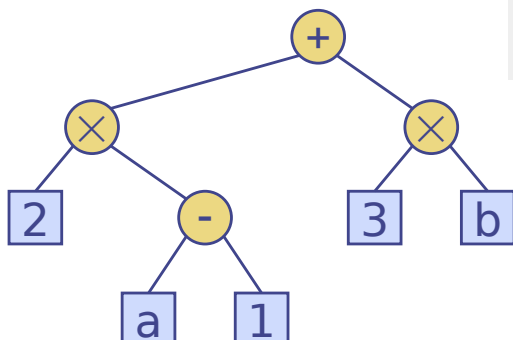
Postorder: 11 17 22 21 25 28 27 23 47 45 65 54 43

Inorder: 11 17 21 22 23 25 27 28 43 45 47 54 65

Print Arithmetic Expressions

Specialization of an inorder traversal

- print operand or operator when visiting node
- print "(" before traversing left subtree
- print ")" after traversing right subtree



```
15 def inorderTraversal(root):
16     if root:
17         if root.left:
18             print("(", end="")
19
20         inorderTraversal(root.left)
21         print(root.data, end="")
22         inorderTraversal(root.right)
23
24         if root.right:
25             print(")", end="")
```

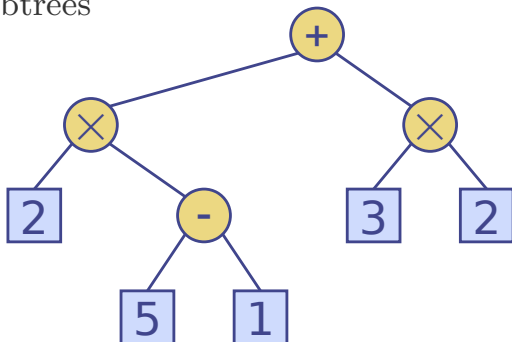
$((2 \times (a - 1)) + (3 \times b))$

Evaluate Arithmetic Expressions

Specialization of a postorder traversal

recursive method returning the value of a subtree

when visiting an internal node, combine the values of the subtrees



Algorithm *evalExpr(v)*

if *is_leaf*(v)

return *v.element* ()

else

x $\leftarrow$ *evalExpr*(*left* (v))

y $\leftarrow$ *evalExpr*(*right* (v))

$\diamond \leftarrow$ operator stored at v

return *x* $\diamond$ *y*

29

Applications of various Tree traversals

Traversal Type	Applications
Pre-Order	* Expression Trees: Prefix notation (Polish notation)
	* Serialization/Deserialization of Trees
	* Copying Trees: Creates a deep copy by visiting root before children
	* Directory Structure Traversal: Processes current directory before subdirectories Eg. common in directory listing operations, backup processes, and recursive file searches
In-Order	* Binary Search Trees (BST): Retrieves nodes in sorted order, ideal for printing or validating BSTs
	* Expression Trees (Infix Notation): Produces infix expressions (human-readable)

Applications of various Tree traversals

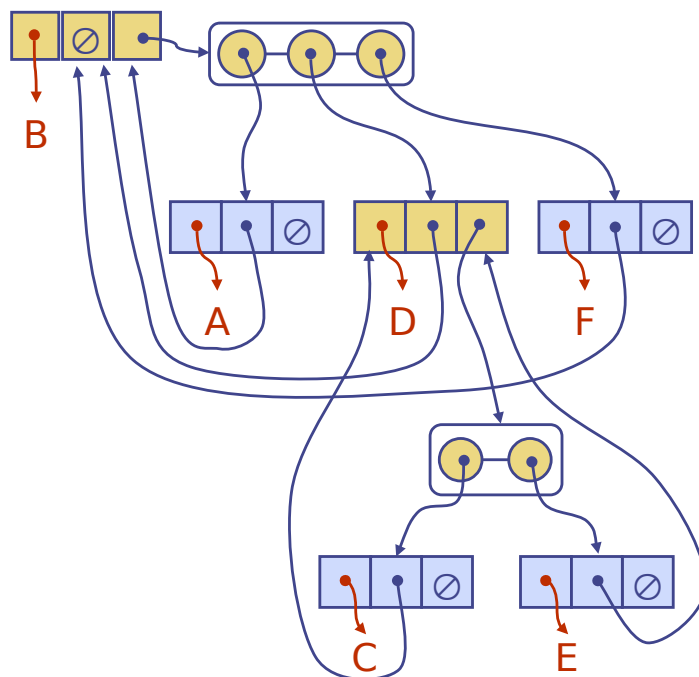
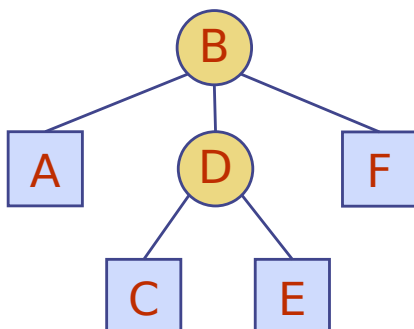
Traversal Type	Applications
Post-Order	* Deleting or Freeing Nodes in a Tree: Processes child nodes before the parent, suitable for deallocation
	* Expression Trees (Postfix Notation): Yields postfix notation, useful for postfix evaluation
	* Dependency Resolution: Ensures prerequisites are processed before the parent task. Eg. in Linux package installation
	* File System Management: Processes subdirectories/files before the parent. Eg. useful deleting directories or computing the total size of a directory (including all its contents).

Linked Structure for Trees

A node is represented by an object storing

- Element
- Parent node
- Sequence of children nodes

Node objects implement the Position ADT



Running times of the accessor methods of an n-node general tree implemented with a linked structure.

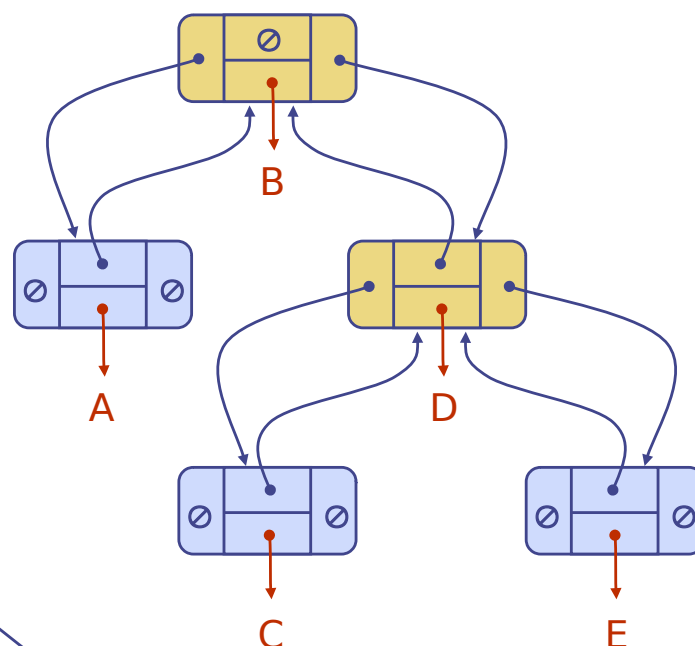
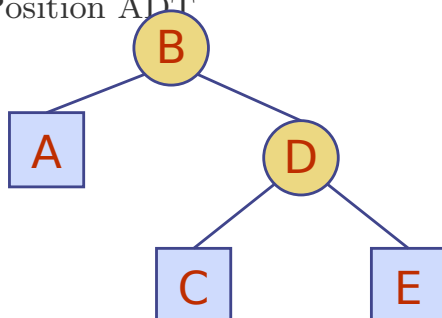
Operation	Running Time
len, is_empty	$O(1)$
root, parent, is_root, is_leaf	$O(1)$
children(p)	$O(c_p + 1)$
depth(p)	$O(d_p + 1)$
height	$O(n)$

Linked Structure for Binary Trees

A node is represented by an object storing

- Element
- Parent node
- Left child node
- Right child node

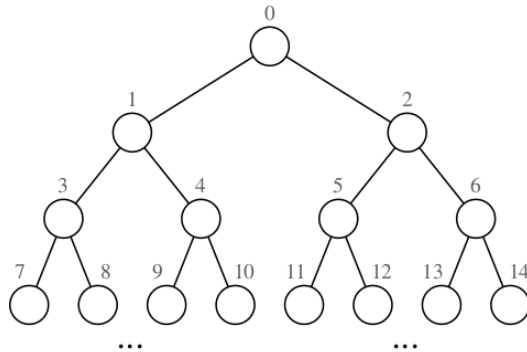
Node objects implement the Position ADT



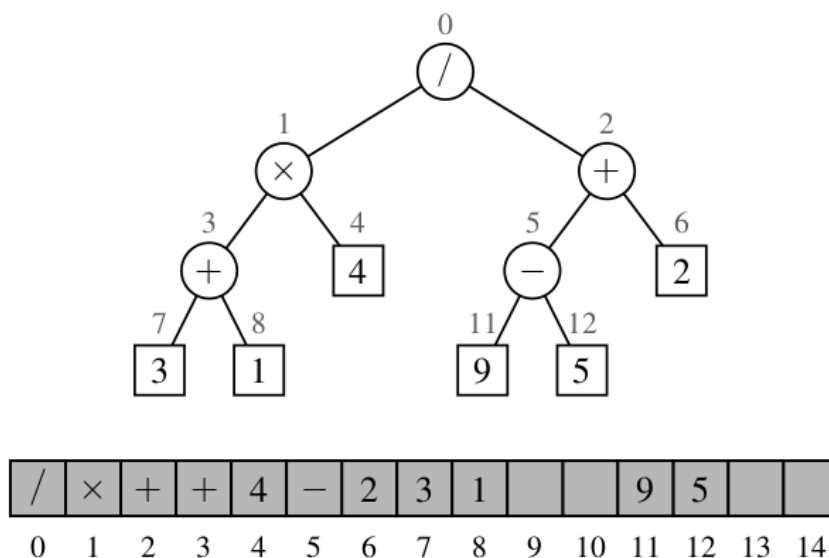
Array-Based Representation of a Binary Tree

- If p is the root of T , then $f(p) = 0$.
- If p is the left child of position q , then $f(p) = 2f(q) + 1$.
- If p is the right child of position q , then $f(p) = 2f(q) + 2$.

The numbering function f is known as a **level numbering** of the positions in a binary tree T , for it numbers the positions on each level of T in increasing order from left to right.



Array-Based Representation of a Binary Tree



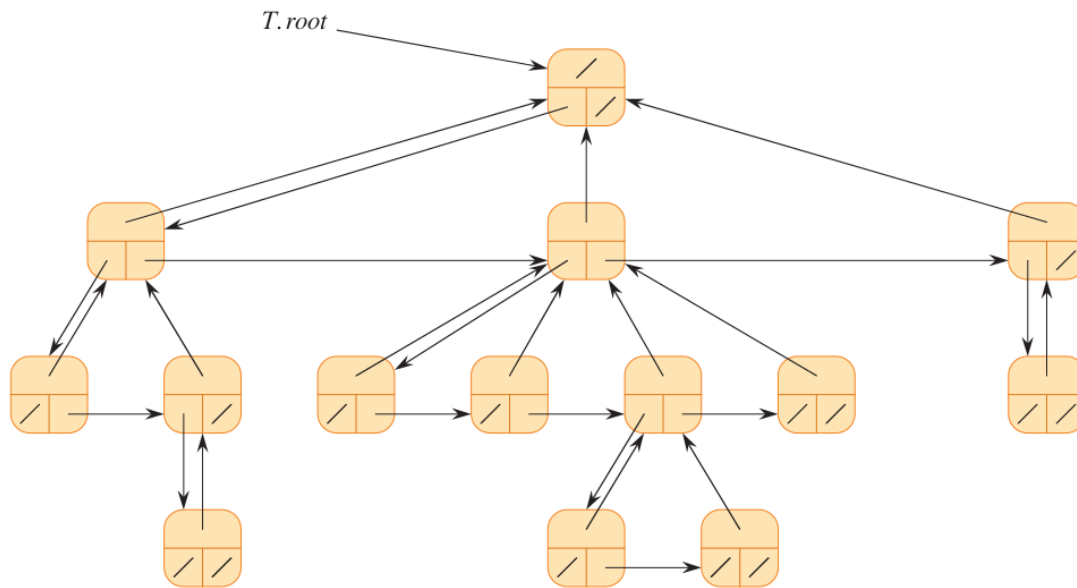
Breadth-First Tree Traversal

**while** Q not empty **do**

perform the “visit” action for position p

Q.enqueue(c) {add p's children to the end of the queue for later visits}

left-child, right-sibling representation of a tree



A scheme to represent trees with arbitrary numbers of children.

left-child, right-sibling representation

Merits:

Memory efficiency: By limiting references per node to two, LCRS saves memory compared to storing all children directly.

Simpler coding: It leverages the binary tree structure, which is often easier to work with than complex multi-way tree structures.

Demerits:

Slower access: Finding specific children by index can be slower as you need to traverse through siblings to reach the desired one.

Complex Tree Traversals: Some tree traversals, like level-order traversal, might require more complex algorithms compared to other representations.

About yield statement in Python

```
def even_numbers():
    i = 0
    while True:
        yield i
        i += 2

# print the first 6 even numbers
for n in even_numbers():
    if n > 10:
        break
    print(n)
```

A function with a yield statement in Python is called a generator function. When called, it returns a generator object, which is an iterator that produces values on demand.

Benefits: Memory Efficiency, Lazy Evaluation, can be used as an iterator (without any index)

Exercise problems

Let T be an n -node binary tree that may be improper. Describe how to represent T by means of a proper binary tree T with $O(n)$ nodes.

What are the minimum and maximum number of internal and external nodes in an improper binary tree with n nodes?

Exercise problems

- What is the minimum number of external nodes for a proper binary tree with height h ? Justify your answer.
- What is the maximum number of external nodes for a proper binary tree with height h ? Justify your answer.
- Let T be a proper binary tree with height h and n nodes. Show that

$$\log(n+1) - 1 \leq h \leq (n-1)/2.$$

- For which values of n and h can the above lower and upper bounds on h be attained with equality?

Exercise problems

Draw a binary tree T that simultaneously satisfies the following:

- Each internal node of T stores a single character.
- A *preorder* traversal of T yields EXAMFUN.
- An *inorder* traversal of T yields MAFXUEN.

In a nonempty proper binary tree T , with n_E external nodes and n_I internal nodes, we have $n_E = n_I + 1$.

Prove by Induction

Exercise problems

For a tree T , let n_I denote the number of its internal nodes, and let n_E denote the number of its external nodes. Show that if every internal node in T has exactly 3 children, then $n_E = 2n_I + 1$.

Define the **internal path length**, $I(T)$, of a tree T to be the sum of the depths of all the internal positions in T . Likewise, define the **external path length**, $E(T)$, of a tree T to be the sum of the depths of all the external positions in T . Show that if T is a proper binary tree with n positions, then $E(T) = I(T) + n - 1$.

Exercise problems

Let T be a (not necessarily proper) binary tree with n nodes, and let D be the sum of the depths of all the external nodes of T . Show that if T has the minimum number of external nodes possible, then D is $O(n)$ and if T has the maximum number of external nodes possible, then D is $O(n \log n)$.

The postorder traversal of a binary tree is 8,9,6,7,4,5,2,3,1. The inorder traversal of the same tree is 8,6,9,4,7,2,5,1,3. The height of a tree is the length of the longest path from the root to any leaf. The height of the binary tree above is _____.

Exercise problems

Consider a binary tree T that has 200 leaf nodes. Then the number of nodes in T that have exactly two children are _____.

The preorder traversal of a binary search tree is 15, 10, 12, 11, 20, 18, 16, 19. Which one of the following is the postorder traversal of the tree?

- a) 20, 19, 18, 16, 15, 12, 11, 10
- b) 10, 11, 12, 15, 16, 18, 19, 20
- c) 19, 16, 18, 20, 11, 12, 10, 15
- d) 11, 12, 10, 16, 19, 18, 20, 15

References

Data Structures and Algorithms in Python

Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser

Introduction to Algorithms

Leiserson, Stein, Rivest, Cormen

Algorithms, 4th Edition

Robert Sedgewick and Kevin Wayne

Few Images from the internet